



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Cloud-Aware Learning-Based Attitude Control for Rainforest Earth Observation

Semester Project

MSc Mechanical Engineering, ETH Zürich

Cédric Grieder

supervised by Prof. Dr. Marco Hutter (ETH RSL); Alejandro Torrents Rufas & Mathias
Burkhalter (Beyond Gravity)

Robotic Systems Lab, ETH Zürich | Semester project at Beyond Gravity

July 9, 2026

Abstract

Satellite imaging of tropical rainforests is limited by persistent cloud cover. Conventional controllers image every target on a fixed pointing schedule, regardless of cloud conditions. Shutter budget and downlink capacity are wasted on cloud-obscured frames scientists cannot use. An autonomous pointing and scheduling policy that reacts to onboard camera observations would use those resources more efficiently.

Because that idea appears straightforward, this work asks a narrower question: what reward shaping and action-space design are required to train such a policy in simulation? We build a reproducible platform for that investigation: single-axis attitude dynamics, ray-tracing cameras, parameterised cloud primitives, and a multi-phase safety controller. A deterministic sequential-target overflight policy serves as the comparison baseline. Two off-policy reinforcement-learning algorithms are trained and evaluated: Soft Actor-Critic and Maximum a Posteriori Policy Optimisation.

The experiments show that the problem is not trivial in practice. Reward structure, how the agent commands attitude, and correct algorithm implementation each had to be resolved before useful behaviour emerged. Both algorithms eventually develop selective shuttering absent in the deterministic baseline, but reliable mission-level gains over that baseline were not established within the semester scope.

The study is limited to a 2D single-pass orbital model. Attitude safety constraints are enforced throughout. Flight hardware, multi-orbit planning, and three-dimensional operations are out of scope.

Contents

Abstract	2
Nomenclature	5
0.1 Abbreviations	5
0.2 Symbols	5
0.3 Physical and simulation constants	5
1 Introduction	7
1.1 Motivation	7
1.2 Research question	8
1.3 Deterministic baseline	8
1.4 Scope and simplifications	9
1.5 Contributions	9
1.6 Report structure	10
2 Methods	10
2.1 Environment overview	10
2.2 Simplifications	11
2.3 Dynamics model	11
2.4 Overflight and observation model	12
2.5 Baseline strategy	14
2.6 Attitude safety controller	14
2.7 Reward design	14
2.8 Learning algorithm: MPO	15
2.9 Observation and action space	16
2.10 Simulation architecture	17
3 Experiments	17
3.1 Master comparison table	18
3.2 Exp 1: Shutter threshold ablation	18
3.3 Exp 2: Modular encoder ablation	19
3.4 Exp 3: SAC vs. MPO algorithm comparison	19
3.5 Exp 4: Vector pointing mode	19
3.6 Exp 5: MPO network width ablation	20
3.7 Exp 7: SAC vector + budget-exhausted shutter penalty	20
3.8 Exp 8: MPO decoupled-KL dual fix (torque mode)	20
3.9 Exp 9: SAC shutter reward split (waste penalty off)	21
3.10 Exp 10: MPO safe-mode penalty	21
3.11 Exp 11: MPO fixed dual, vector mode	21
3.12 Exp 12: MPO vector + torque-effort penalty ablation	22
3.13 Exp 13: MPO learning-cadence sweep	22
3.14 Exp 14: MPO multi-environment target selection	22
4 Results	22
4.1 Aggregate performance comparison	23
4.2 Learning progression	23
4.3 Shutter spam and budget efficiency	24
4.4 Safe-mode and safety overhead	24
4.5 Algorithm comparison: SAC vs. MPO	25

5	Discussion	25
5.1	What the experiment sequence taught	25
5.2	Answer to the research question	25
5.3	Control hierarchy and action interface	26
5.4	Rejected paths	26
5.5	Limitations	27
5.6	Deferred extensions	28
6	Conclusion	28
A	Supplementary material	28
A.1	Repository	28
A.2	Source index	29
A.3	Live parameter record	29
A.4	Artifact paths	29

Nomenclature

0.1 Abbreviations

Abbreviation	Definition
CNN	Convolutional neural network (vision observation encoder)
EO	Earth observation
FOV	Field of view
GSD	Ground sample distance
LLA	Latitude, longitude, altitude (geodetic coordinates)
LOS	Line of sight
MLP	Multilayer perceptron (scalar observation branch)
SAC	Soft Actor-Critic
MPO	Maximum <i>a posteriori</i> policy optimization
OBC	On-board computer
PD	Proportional-derivative (pointing controller)
RL	Reinforcement learning
RW	Reaction wheel
WGS84	World Geodetic System 1984 (reference ellipsoid)

0.2 Symbols

Symbol	Meaning
R_E	Mean Earth radius (orbital geometry)
μ	Earth gravitational parameter
h	Orbit altitude above mean Earth radius
I_{sat}	Spacecraft moment of inertia (2D model: I_{zz})
$\boldsymbol{\tau}$	Reaction-wheel torque command [N m] (scalar in the 2D kernel)
θ_{orb}	True anomaly of the satellite (= orbit-plane angle in circular orbit)
θ_{body}	Body z -axis angle in the orbit disk
$\omega_{\text{sat}}, \omega_{\text{w}}$	Spacecraft body rate and wheel speed
f_{cloud}	Primary-camera cloud-blocked fraction along the sensor strip

0.3 Physical and simulation constants

Unless noted, values are fixed inputs; h and cloud layouts are randomized per episode (Table 3).

Table 1: Earth, orbit, and spacecraft parameters.

Quantity	Symbol	Value	Unit / note
Mean Earth radius	R_E	6371.0088	km
Gravitational parameter	μ	398 600.4418	km^3/s^2
Geodetic model	—	WGS84 ellipsoid (surface / LOS)	
Orbit altitude (episode)	h	uniform on [510 km, 570 km]	
Satellite mass	m_{sat}	250	kg
Moment of inertia (3D)	I_{xx}, I_{yy}, I_{zz}	16.6, 21.7, 31.2	kg m^2
Moment of inertia (2D sim.)	I_{sat}	31.2	kg m^2 (I_{zz})
RW max torque	τ_{max}	0.1	N m
RW max momentum	H_{max}	0.4	N m s
Star-tracker rate limit	—	3	$^\circ/\text{s}$
Off-nadir hard limit	—	45	$^\circ$

Table 2: Camera, simulation timing, and sensing discretization.

Quantity	Symbol	Value	Unit / note
Sensor resolution	—	9344×7000 pixels (cross $\times$ along)	
Pixel pitch	—	3.2	μm
Focal length	f	1067	mm
Exposure time	t_{exp}	100	μs
Observation-line bins	—	101	along-track discretisation
Simulation step	Δt	1.5	s
Episode step cap	—	1000	steps
Max captures per orbit	—	10	shutter budget (resource proxy)
<i>Secondary wide-angle camera</i>			
Sensor resolution	—	5312×2988 pixels	
Pixel pitch	—	1.55	μm
Vertical FOV	—	70	$^\circ$
Boresight tilt from nadir	—	25	$^\circ$
Observation-line bins	—	200	along-track discretisation

Table 3: Mission profile and reward scales (default training configuration).

Quantity	Value / range
Target grid	50 areas, 15 km extent, 40 km spacing along corridor
Target anchor latitude	80.0°N (grid start)
Cloud patches per episode	uniform integer in [15, 30] along corridor
Cloud horizontal extent	1 km to 80 km
Cloud base altitude	4 km to 12 km
Cloud thickness	1 km to 16 km; top capped at 20 km
Capture reward weight	$w_{\text{capture}} = 100$
Optimal / threshold ground range	$d_{\text{op}} = 400$ km, $d_{\text{th}} = 1500$ km
Viewing-distance outer gate	2500 km

1 Introduction

1.1 Motivation

Climate and environmental science depend on repeat optical imaging of regions such as tropical rainforests, where cloud cover is frequent and unpredictable at the timescale of a satellite pass. Mission operators today often rely on deterministic strategies with fixed nadir attitude and pre-planned shutter times that treat every target along a ground track equally. When clouds occlude the line of sight, those strategies still consume onboard memory and downlink budget on low-value frames, while scientists lose observations that may not be recoverable on the next pass. The cost is therefore threefold: reduced science yield, wasted capture capacity, and unnecessary actuator activity from indiscriminate pointing and slewing.

Autonomous onboard decision-making is motivated by latency: ground-in-the-loop replanning cannot react quickly enough as cloud fields evolve during overflight. Both *when* to expose the shutter and *how* to orient the spacecraft for maximal image quality therefore matter, and they must respect attitude safety limits on reaction-wheel use and pointing envelopes. From the outside, the required behaviour looks straightforward: use onboard camera feedback to skip cloud-occluded targets and capture only when the view is clear. This report treats that apparent simplicity as the starting point and asks what training design is actually required to reach it.

The specific orbit geometry is not tied to a polar versus equatorial regime. Any circular orbit viewed at the instant of overflight is equivalent for the control problem posed here.

Current operational process. In a typical Earth-observation mission today, domain scientists compile a list of target areas they wish to image and submit it to mission control. The ground segment then builds an observation schedule: it checks which targets fall within the satellite’s ground track on upcoming passes, verifies observable constraints such as solar illumination and off-nadir angle limits, and uplinks a fixed command sequence. This process works well for constraints that can be predicted from orbital mechanics. Cloud cover, however, cannot be scheduled out: the weather state at the moment of overflight is unknown at planning time, and the uplinked schedule cannot be modified during the pass. The satellite therefore executes its pre-planned capture sequence regardless of what the cameras actually see, consuming onboard storage, electrical power, downlink capacity, and attitude-control effort on frames that may be entirely cloud-occluded [3].

1.2 Research question

Cloud-aware autonomous scheduling is an obvious improvement over fixed pre-planned capture. The motivating question of this report is therefore not whether such a policy should exist, but how to train one in simulation:

What reward shaping and action-space design are sufficient for a reinforcement-learning agent to learn selective pointing and shuttering during a cloud-occluded Earth-observation pass?

Capture decisions are sequential and uncertain. Cloud occlusion, attitude dynamics, and discrete shutter events interact over a single pass, so the design of the learning interface matters as much as the algorithm choice. The experiments below treat this as a design-space question: which training choices make selective shuttering learnable, and which do not.

1.3 Deterministic baseline



Fig. 0: Baseline overflight, three phases. Overhead 2D view of one deterministic baseline episode. Phase 1 **Approach**: satellite enters the target corridor LOS window. Phase 2 **Point**: OBC PD loop slews toward target bearing; off-nadir angle grows. Phase 3 **Capture**: boresight error drops below threshold (Table 4), shutter fires. Repeat for each scheduled target regardless of cloud cover.

The reference strategy is a pre-scheduled sequential controller used as the comparison anchor throughout the experiments (Section 2.5). An onboard computer (OBC) runs a proportional–derivative pointing loop: it commands a target body angle (nadir coast or target bearing) and converts the tracking error into a reaction-wheel torque request. The same attitude-safety stack and simulation kernel as the learned policy apply; safety overrides are not disabled.

Table 4 summarises the baseline scheduling and tuning. The policy has no access to cloud information and cannot adapt its capture list.

Table 4: Deterministic baseline configuration (comparison anchor).

Aspect	Setting
Scheduling logic	Strided target list: 10 captures per pass from a 50-target corridor (indices 0, 5, 10, ..., 45 on the evaluation seed). Nadir coast between engages; slew to target bearing when the orbit-angle window opens.
OBC pointing	PD loop on body-angle tracking error; gains $k_p \approx 1.15 \text{ N m/rad}$, $k_d \approx 12.0 \text{ N ms/rad}$ derived from $\tau_{\max} = 0.1 \text{ N m}$, $I_{\text{sat}} = 31.2 \text{ kg m}^2$, and a 5° deceleration band.
Boresight lock threshold	15° between body boresight and target bearing (shutter fires once locked, or if image quality ≥ 0.25 while the target is in the sensor strip).
Lead margin	20° orbit-angle margin before target engage window opens.
Shutter commands	10 per episode (one per scheduled target; matches the episode shutter budget below).
Cloud awareness	None (captures on schedule regardless of occlusion).

A real mission is additionally constrained by onboard memory, downlink bandwidth, and energy [3]. We model this resource limit with a *shutter budget*: a hard cap of ten capture actions per episode. Because the deterministic baseline treats every scheduled target equally, that budget is consumed indiscriminately across clear and cloud-occluded targets alike. This yields a fixed, predictable observation cadence but wastes capture capacity on frames that are scientifically unusable when a cloud is in the field of view.

1.4 Scope and simplifications

The model deliberately stays close to the control problem while omitting much operational detail: 2D attitude dynamics, basic circular-orbit geometry, physical ray tracing for cameras and clouds, safety arbitration, and single-pass episodes. It does not model multi-orbit scheduling, flight hardware, or real imagery ingestion. Illumination conditions are not modelled: all episodes are treated as daytime passes with perfect solar lighting. This is a deliberate simplification. Optical Earth observation is not performed at night, and solar geometry only excludes certain orbit geometries rather than driving frame-to-frame image quality during a pass. Cloud occlusion is a far stronger and more variable image quality constraint within a single pass, and is the focus of this work. Section 2 describes how the question was investigated and lists modelled subsystems explicitly; Discussion states extensions (three-dimensional attitude, richer terrain, stronger RL benchmarks) as logical next steps rather than deliverables of this semester project.

Long-term context. The intended onboard stack is hierarchical: a mission planner sets observation intent, a learned policy decides when and what to image, and an OBC executes maneuvers through PD pointing under hard attitude-safety limits. This report maps the learning interface for that scheduling layer, not the full flight stack. Development began with direct reaction-wheel torque commands; review with system engineers after aggressive simulated slewing motivated modelling an attitude-safety arbitrator on every agent action (Section 2.6). The experiment sequence compares torque and nadir-relative pointing (*vector*) modes; vector mode delegates rate control to the OBC and became the default from Exp 11 onward. Section 5.3 expands the rationale for segregating scheduling from low-level control.

1.5 Contributions

This report documents:

- **Platform.** A reproducible simulation-and-learning pipeline with seeded mission randomization, canonical episode rollouts, matched baseline replay, and view-only rendering so that reward and action-space variants can be compared under identical cloud and target draws.
- **Design-space study.** A sequence of fourteen experiments that maps which reward terms, action representations, and algorithm choices enable cloud-aware shuttering behaviour, and which do not.
- **Evaluation protocol.** Warmup, training, and evaluation conventions with a fixed environment seed and a deterministic baseline as the comparison anchor.
- **Evidence.** Qualitative and quantitative artefacts from frozen runs, including selective-shuttering behaviour in late-training episodes where learning succeeded.

The engineering platform and the experimental map are the primary deliverables; mission-level outperformance of the baseline was not established within the semester scope (Section 5).

1.6 Report structure

Section 2 describes the research approach, simulation environment, and learning method. Section 3 documents the design-space search: which reward and action-space choices were tested and what each revealed. Section 4 aggregates the behavioural and return outcomes. Sections 5 and 6 interpret the mapped design space, rejected shortcuts, and outlook. Technical constants are listed in the Nomenclature section; source-code entry points are in the Appendix.

2 Methods

This section describes how the research question (Section 1.2) was investigated. We built a simulation platform and ran a sequence of controlled experiments that varied reward terms, action representation (direct torque versus nadir-relative pointing offset), and learning algorithm. Each configuration was evaluated against the deterministic baseline (Section 1.3) under matched environment draws: same altitude, cloud layout, and target positions per episode. Success is measured by whether the agent learns a stable policy that uses vision feedback to shutter selectively rather than on every step.

Alongside the design-space map, the work pursued three practical deliverables:

- a reproducible train-eval pipeline with seeded missions and matched baseline comparison;
- documentation of which reward and action-space choices enable learning and which do not;
- demonstration exports (baseline versus trained-policy videos, learning curves, and comparison plots) for behavioural assessment.

The subsections below document every modelling choice, constant, and algorithm in sufficient detail to reproduce the experiment results. Numeric values appear in the Nomenclature and in the tables below; repository source locations are listed in the Appendix.

2.1 Environment overview

The mission is a single-satellite nadir-looking Earth-observation overflight. One episode represents a single contiguous pass over a ground corridor.

Altitude and orbit. The simulation uses a 2D orbit-plane model: the satellite moves as a point on a circle whose true anomaly θ_{orbit} advances at the instantaneous circular orbit rate (in a circular orbit, true anomaly equals orbital angle). Orbital altitude is sampled uniformly from [510 km, 570 km] per episode; all other geometry scales with altitude, so the choice of specific ground track is arbitrary.

Target corridor. Fifty observation targets are placed at fixed positions along a ground corridor directly below the orbit plane. Their angular positions in the orbit plane are computed once at episode initialisation from the sampled altitude. The specific geographic location of the corridor is irrelevant: the agent only ever sees orbit-plane angles, cloud fractions, and attitude state.

Episode window. The simulation arc is derived from the target-area geometry. First, the orbit-plane θ offsets that place the satellite’s line-of-sight on the near and far edges of the full target corridor are computed analytically from the sampled altitude and target layout, padded by a contact-margin angle of 0.05° so the satellite can already see the first target at episode start. Second, the satellite is flown only through the central 70% of this padded arc: the outer margins are outside the attitude-safety reach and contribute only dead compute. At the canonical

simulation timestep of $\Delta t = 1.5\text{ s}$ this yields approximately 516 controller steps per episode ($\approx 774\text{ s}$).

Simulation timestep selection (H0 sweep). Choosing Δt requires balancing three competing constraints:

1. **Controller authority:** Δt must be small enough that the low-level attitude controller can still respond to the reaction-wheel dynamics within one step.
2. **Agent decisiveness:** the agent must be able to take meaningful pointing and shutter actions on a timescale relevant to the target encounter windows.
3. **Reward signal fidelity:** undersampling the target overflight would collapse the sparse capture-credit signal, starving the replay buffer of useful transitions.

To find the largest Δt satisfying these constraints we ran a parity sweep (Exp. 0, Phase H0) over four candidates: $\Delta t \in \{0.4, 0.8, 1.0, 1.5\}\text{ s}$, with controller update interval matched to Δt . Each candidate ran one baseline warmup episode and had to satisfy: $n_{\text{shutter}} > 0$ and episode return $\geq 50\%$ of the 0.4 s reference. All four candidates passed. $\Delta t = 1.5\text{ s}$ was selected as the largest parity-passing timestep, reducing the per-episode integration cost from ~ 1935 integration states to ~ 516 controller steps and proportionally increasing training throughput.

2.2 Simplifications

Table 5 summarises which subsystems are modelled and which are deliberately omitted.

Table 5: Modelled vs. not-modelled subsystems.

Subsystem	Modelled	Not modelled
Orbit propagation	Circular 2D (disk model)	J_2 , atmospheric drag, eclipse
Attitude dynamics	2D rigid body, reaction-wheel torque chain	3-axis, flexibility, RW friction
Camera	1D ray tracing, image quality (motion smear)	Radiometric calibration, TDI, MTF
Cloud occlusion	Parameterised cloud-arc primitives	Spatially correlated fields, multi-layer
Terrain	Spherical Earth (WGS84 for geodesy only)	DEM, shadow, surface reflectance
Multi-orbit mission	Single pass per episode	Revisit scheduling, downlink windows
Hardware latency	Not modelled	Onboard CPU budget, comms delay

The 2D simplification is the most consequential: all attitude variables are scalar (θ_{body} , ω_{sat} , ω_{wheel}), and out-of-plane dynamics are absent. This is consistent with the scope stated in the introduction and is a known limitation (Section 5.5).

2.3 Dynamics model

Equations of motion. The satellite body angle θ_{b} and inertial rate ω_{sat} evolve under a single-axis reaction-wheel coupling:

$$\dot{\omega}_{\text{sat}} = -\frac{\tau_{\text{wheel}}}{I_{\text{sat}}}, \quad \dot{\omega}_{\text{wheel}} = \frac{\tau_{\text{wheel}}}{I_{\text{wheel}}}, \quad \dot{\theta}_{\text{b}} = \omega_{\text{sat}}, \quad (1)$$

where $I_{\text{sat}} = 31.2\text{ kg m}^2$ (Table 1) and the wheel inertia $I_{\text{wheel}} = \tau_{\text{max,RW}}/\omega_{\text{w,max}} = 0.1\text{ N m}/150\text{ rad s}^{-1}$. Both are integrated forward with forward-Euler at $\Delta t_{\text{sim}} = 1.5\text{ s}$ (Table 2).

Reaction-wheel limits. The wheel applies a torque $|\tau_{\text{wheel}}| \leq \tau_{\text{max,RW}} = 0.1 \text{ N m}$ and stores angular momentum up to $H_{\text{max}} = 0.4 \text{ N m s}$. A low-level rate gate blocks commands that would accelerate the satellite body beyond the star-tracker rate limit of $\dot{\theta}_{\text{max}} = 3^\circ \text{ s}^{-1}$; decelerating torques are always passed.

Satellite mass. Bus mass is $m_{\text{sat}} = 250 \text{ kg}$ (relevant only for orbit-speed computation; inertia is the primary dynamics parameter).

2.4 Overflight and observation model

Camera geometry. The primary nadir-looking camera parameters are listed in Table 2. The vertical field of view and ground-sampling distance (GSD) are computed analytically from sampled altitude using a pinhole model. At 540 km the nominal GSD is approximately 1.6 m.

Ray discretisation. At each simulation timestep the camera boresight is cast as 101 uniform rays spanning the vertical FOV along the primary observation line. A secondary wide-angle camera (Table 2) provides an additional 200-bin observation line. Both lines are evaluated with the same ray-intersection kernel each step.

Image quality and motion smear. Capturing a useful image requires the boresight ground footprint to remain nearly stationary during the exposure window. The full derivation proceeds in three steps: (i) compute the boresight ground velocity $v_{\text{boresight}}$ from orbit and body rates; (ii) convert to a physical ground blur δ during exposure; (iii) map δ to a scalar quality score $q \in [0, 1]$.

Step 1: Boresight ground speed. In the 2D orbit-disk model the satellite position is

$$\vec{s} = r_{\text{orb}} [\cos \theta_{\text{orb}}, \sin \theta_{\text{orb}}]^\top, \quad (2)$$

with orbital radius r_{orb} and true anomaly $\theta_{\text{orb}}(t)$ advancing at angular rate $\dot{\theta}_{\text{orb}} = \omega_{\text{orb}}$. The boresight unit vector in the disk plane is

$$\hat{d} = [\cos \phi, \sin \phi]^\top, \quad \dot{\phi} = \omega_{\text{body}}, \quad (3)$$

where ϕ is the absolute boresight angle (body angle plus nadir direction) and $\omega_{\text{body}} = \dot{\theta}_{\text{body}}$ is the current body rate.

The boresight ray $\vec{s} + t \hat{d}$ intersects the WGS84 oblate spheroid ($a = 6\,378\,137 \text{ m}$, $b = 6\,356\,752 \text{ m}$) at slant distance t^* from the positive root of

$$A t^2 + B t + C = 0, \quad (4)$$

where

$$A = \frac{d_x^2 + d_y^2}{a^2} + \frac{d_z^2}{b^2}, \quad (5)$$

$$B = 2 \left(\frac{s_x d_x + s_y d_y}{a^2} + \frac{s_z d_z}{b^2} \right), \quad (6)$$

$$C = \frac{s_x^2 + s_y^2}{a^2} + \frac{s_z^2}{b^2} - 1. \quad (7)$$

(All 3-D coordinates are obtained by embedding the 2D disk in the (x, z) ECEF plane.)

The boresight ground hit is $\vec{g} = \vec{s} + t^* \hat{d}$. Differentiating implicitly with respect to time:

$$\dot{t}^* = -\frac{\dot{A}(t^*)^2 + \dot{B}t^* + \dot{C}}{2At^* + B}, \quad (8)$$

where $\dot{A}$, $\dot{B}$, $\dot{C}$ are computed from $\dot{\vec{s}}$ (satellite velocity) and $\dot{\hat{d}}$ (boresight rotation rate):

$$\dot{\vec{s}} = r_{\text{orb}} \omega_{\text{orb}} [-\sin \theta_{\text{orb}}, 0, \cos \theta_{\text{orb}}]^\top, \quad \dot{\hat{d}} = \omega_{\text{body}} [-\sin \phi, 0, \cos \phi]^\top. \quad (9)$$

The hit velocity is then

$$\dot{\vec{g}} = \dot{\vec{s}} + \dot{t}^* \hat{d} + t^* \dot{\hat{d}}. \quad (10)$$

The boresight ground speed is $v_{\text{bore}} = \|\dot{\vec{g}}\|$.

Step 2: Ground blur during exposure. During a finite exposure $t_{\text{exp}} = 100 \mu\text{s}$ the boresight footprint moves

$$\delta = v_{\text{bore}} t_{\text{exp}} \quad [\text{m}]. \quad (11)$$

The pixel smear in GSD units is

$$\sigma = \frac{\delta}{\text{GSD}_{\text{eff}}}, \quad (12)$$

where the effective GSD at off-nadir angle α is $\text{GSD}_{\text{eff}} = \text{GSD}_0 / \cos \alpha$, with nadir GSD

$$\text{GSD}_0 = \frac{p \cdot h}{f} = \frac{3.2 \mu\text{m} \cdot h}{1067 \text{ mm}} \quad (13)$$

(pixel pitch p , altitude h , focal length f). At $h = 540 \text{ km}$: $\text{GSD}_0 \approx 1.62 \text{ m}$, giving a nadir swath of $9344 \times 1.62 \text{ m} \approx 15.1 \text{ km}$ along-track.

Step 3: Normalised quality score. Rather than using smear in pixels (which depends on altitude), quality is expressed in physical ground units via a Lorentzian:

$$q = \frac{r}{\delta + r}, \quad r = 0.30 \text{ m}, \quad t_{\text{exp}} = 100 \mu\text{s}. \quad (14)$$

Properties: $q \rightarrow 1$ as $\delta \rightarrow 0$ (perfect tracking); $q = 0.5$ at $\delta = r = 0.30 \text{ m}$; q is indifferent to GSD (quality degrades from optical smear, not pixel count).

Characteristic regimes. At the canonical orbit ($h = 540 \text{ km}$, $\omega_{\text{orb}} \approx 1.08 \times 10^{-3} \text{ rad s}^{-1}$):

- **Nadir, zero body rate:** $v_{\text{bore}} = r_{\text{orb}} \omega_{\text{orb}} \approx 7.6 \text{ km s}^{-1}$, $\delta \approx 0.76 \text{ m}$, $q \approx 0.28$.
- **Target-tracking** ($\omega_{\text{body}} = \omega_{\text{orb}}$): $v_{\text{bore}} \approx 0$, $\delta \approx 0$, $q \approx 1.0$.
- **Half-tracking:** $\omega_{\text{body}} = 0.5 \omega_{\text{orb}}$, $\delta \approx 0.38 \text{ m}$, $q \approx 0.44$.

The quality factor directly enters the capture-credit reward (Eq. 16), giving the agent an incentive to slow the boresight ground motion by pointing toward and tracking each target across its observation window.

Cloud model. Cloud primitives are parameterised arcs in the orbit plane. For each episode a seeded generator places cloud patches along the target corridor with extent, altitude, and thickness drawn from the ranges in Table 3. The cloud-blocked fraction of each camera-ray bin is computed at every timestep and contributed to the per-step observation and reward.

2.5 Baseline strategy

The reference controller is a deterministic sequential-target overflight policy (Table 4). It issues a pre-planned sequence of pointing requests: for each target in orbital order the controller slews to the target bearing when the satellite enters the engagement window (projected off-nadir at the target boresight plus braking distance $\leq 45^\circ$) and fires the shutter once the boresight error is below the 15° lock threshold (Table 4). The baseline uses the same attitude-safety stack and simulation kernel as the learned policy: safety overrides are not disabled and the shutter budget of ten captures per episode is enforced.

The baseline *cannot* observe cloud content: it schedules captures regardless of whether the target is occluded. Its deterministic fixed-cadence shuttering consumes the capture budget equally across clear and cloud-obscured targets, resulting in a fraction of scientifically useless frames. Under the experiment environment this yields a mean evaluation episode return of approximately -38.6 (matched seeds, two evaluation episodes). This value serves as the comparison anchor throughout the design-space study; it is not the quantity the experiments set out to maximise, but the reference against which each reward and action-space variant is measured.

2.6 Attitude safety controller

Every torque command, whether from the learned policy or the baseline, passes through the attitude safety controller before reaching the reaction-wheel plant. The controller operates in two regimes:

Normal-mode taper. In nominal flight the controller applies a linear taper on the commanded torque when the satellite approaches the off-nadir hard limit:

$$\tau_{\text{applied}} = \alpha \tau_{\text{cmd}}, \quad \alpha = \text{clip}\left(\frac{\theta_{\text{hard}} - \theta_{\text{off-nadir}}}{\theta_{\text{hard}} - \theta_{\text{arm}}}, 0, 1\right), \quad (15)$$

where $\theta_{\text{hard}} = 45^\circ$ is the hard limit and θ_{arm} is the kinematic braking distance ($\theta_{\text{arm}} = \theta_{\text{hard}} - \omega^2/(2\tau_{\text{max}}/I_{\text{sat}})$).

Safe-mode entry condition. Safe mode is entered when the off-nadir angle equals or exceeds θ_{hard} , or when the predicted overshoot satisfies $\theta_{\text{off-nadir}} + \theta_{\text{brake}} \geq \theta_{\text{hard}}$.

Safe-mode recovery phases. Once safe mode is active the agent torque command is replaced by an onboard recovery sequence (agent commands are ignored throughout):

1. **BRAKE:** zero-rate command until $|\omega_{\text{sat}} - \omega_{\text{orbit}}| \leq 0.1^\circ \text{ s}^{-1}$.
2. **CRUISE:** slew toward nadir at $\pm 0.35^\circ \text{ s}^{-1}$ until off-nadir $\leq 5.0^\circ$.
3. **SETTLE:** zero-rate command until off-nadir $\leq 1.0^\circ$ and rate $\leq 0.1^\circ \text{ s}^{-1}$.
4. **LOCKOUT:** hold nadir for 60s; agent torque rejected.

After LOCKOUT the controller exits safe mode and normal operation resumes. All phase thresholds are listed in Table 6.

2.7 Reward design

The reward is computed per controller step from the full episode simulation state. All active terms are listed below; inactive terms (distance reward, cloud penalty, energy penalty) were tested in early experiments and disabled by default (see Section 3).

Table 6: Attitude safety controller thresholds.

Parameter	Symbol	Value
Off-nadir hard limit	θ_{hard}	45°
Nadir recovery tolerance	θ_{tol}	1.0°
Decel start (CRUISE→SETTLE)	θ_{decel}	5.0°
Rate stop threshold	ω_{stop}	0.1° s^{-1}
Safe-mode cruise rate	$\dot{\theta}_{\text{crs}}$	$0.35^\circ \text{ s}^{-1}$
Post-recovery lockout	t_{lock}	60 s

Capture quality term (applied at shutter events). When the agent fires the shutter, is within budget, and the target has not been captured before:

$$r_{\text{cap}} = W_{\text{cap}} \cdot \text{cov} \cdot q \cdot (1 - f_{\text{cloud}}), \quad (16)$$

where $\text{cov} \in [0, 1]$ is the fraction of primary camera bins over the dominant target, $q \in [0, 1]$ is the image quality (Section 2.4), $f_{\text{cloud}} \in [0, 1]$ is the cloud-blocked fraction, and W_{cap} is the capture weight. Repeat shutters on an already-captured target consume budget but yield zero applied reward.

Budget-exhausted penalty. When the agent commands a shutter after the 10-capture budget is exhausted: $r_{\text{budget}} = -k_{\text{waste}}$ with $k_{\text{waste}} = 5.0$ (Exp 7, promoted to default).

Shutter-waste penalty (off by default). Optional: when applied capture credit $< \epsilon = 0.001$ on an accepted shutter: $r_{\text{waste}} = -k_{\text{waste}}$. Removed in Exp 9 (see Section 3.9).

Torque-effort regulariser (torque mode only).

$$r_{\text{effort}} = -k_{\text{eff}} \left(\frac{\tau_{\text{cmd}}}{\tau_{\text{max}}} \right)^2, \quad k_{\text{eff}} = 0.1. \quad (17)$$

Applied every step on the agent-requested torque (before safety arbitration); disabled automatically in vector mode.

Safe-mode penalty (Exp 10 only, not default). Per-step penalty while the agent is in safe-mode recovery: $r_{\text{safe}} = -k_{\text{safe}}$, $k_{\text{safe}} = 2.0$. Ruled out at this coefficient (see Section 3.10).

2.8 Learning algorithm: MPO

We use Maximum a Posteriori Policy Optimisation (MPO) [1]. MPO frames policy improvement as a constrained relative-entropy problem solved by alternating E-step and M-step:

E-step (sample-based policy evaluation). Q^π targets are regressed from a target critic; the new non-parametric policy distribution $q^*(a|s) \propto \pi(a|s) \exp(Q^\pi(s, a)/\eta)$ is derived by solving the dual:

$$\eta^* = \arg \min_{\eta > 0} \eta \varepsilon_\eta + \eta \log \mathbb{E}_\pi \left[\exp \left(\frac{Q^\pi(s, a)}{\eta} \right) \right], \quad (18)$$

where $\varepsilon_\eta = 0.1$ is the KL constraint on the temperature dual.

M-step (policy update with trust region). A parametric Gaussian policy π_θ is fitted to $q^\star$ subject to decoupled KL constraints on the mean and variance:

$$\max_{\theta} \mathbb{E}_{q^\star}[\log \pi_\theta(a|s)] \quad \text{s.t.} \quad \text{KL}(\bar{\pi}_\mu \parallel \pi_{\theta,\mu}) \leq \varepsilon_\mu, \text{KL}(\bar{\pi}_\sigma \parallel \pi_{\theta,\sigma}) \leq \varepsilon_\sigma, \quad (19)$$

with Lagrange multipliers $\alpha_\mu, \alpha_\sigma$ updated by dual ascent ($\varepsilon_\mu = 0.1, \varepsilon_\sigma = 0.01$; Table 7).

Decoupled-KL fix (Exp 8). Prior to Exp 8 the implementation used a single coupled KL constraint with a shared multiplier, which led to $\text{KL} \rightarrow 0$ (frozen policy) or $\text{KL} \rightarrow \infty$ (exploding critic) depending on initialisation. The fix implements separate α_μ/α_σ multipliers as specified in Abdolmaleki et al. [1]. All MPO experiments from Exp 8 onwards use the corrected implementation.

Network architecture. Both actor and critic share a 1D-CNN encoder over the camera observation lines (two convolutional layers, kernel size 5, channels [16, 32], stride 2, embedding dimension 32; code embedding dimension 8) followed by fully connected layers (actor: 90 units; critic: 140 units). Hyperparameters are listed in Table 7.

Table 7: MPO hyperparameters (default configuration; all experiments unless noted).

Parameter	Symbol	Value
Batch size	B	256
Replay buffer	$ \mathcal{B} $	50 000
Discount factor	γ	0.99
Target network	τ	0.005
Actor learning rate	η_π	4.5×10^{-4}
Critic learning rate	η_Q	1.0×10^{-3}
Dual learning rate	η_η	1.0×10^{-3}
KL samples (E-step Q)	N_Q	80
KL samples (M-step)	N_π	40
KL temperature constraint	ε_η	0.1
KL mean constraint	ε_μ	0.1
KL variance constraint	ε_σ	0.01
Warmup episodes	—	5
Training episodes	—	50 (except Exp 14: 350)
Eval episodes	—	2
Environment seed	—	7 (Exps 1–13)

2.9 Observation and action space

Observations. The controller observation is assembled at each controller tick from:

- **Attitude scalars:** body angle θ_{body} [rad], satellite body rate ω_{sat} [rad/s].
- **Orbit scalar:** true anomaly θ_{orbit} [rad].
- **Camera lines:** primary 101-bin observation-line code array (integer class per bin: background, target T0, T1, ..., cloud-blocked); secondary 200-bin array.
- **Mission context:** remaining shutter budget (dimensionless count); binary target-captured mask (one entry per target area); signed bearing error toward each target [rad].

Actions. The MPO policy outputs a 2-dimensional continuous action $[a_\tau, a_s] \in [-1, 1]^2$:

- **Torque mode:** $a_\tau \mapsto \tau_{\text{cmd}} = a_\tau \cdot \tau_{\text{max}}$ [N m] (direct RW torque request).
- **Vector mode:** $a_\tau \mapsto f_n = 45^\circ \cdot a_\tau$ (nadir-relative pointing offset); the OBC PD controller converts this to a torque command.
- **Shutter:** $a_s \mapsto (a_s + 1)/2 \in [0, 1]$; boolean shutter command when mapped value > 0.5 .

The torque versus vector mode is a configuration choice at training time; experiments from Exp 11 onward use vector mode as the default.

2.10 Simulation architecture

The simulation, learning, and visualisation layers are strictly separated:

- **Canonical simulation.** One rollout produces the full per-episode world-state time series consumed by training, evaluation, and rendering. No parallel integrators exist.
- **Seeded randomness.** A per-workflow integer seed fixes altitude and cloud layouts; warmup episodes are cached for reproducibility.
- **View-only render.** Visualisation reads a completed simulation time series and performs no numeric propagation.
- **Exclusive training runs.** Only one long training job runs at a time on a given machine, so run artifacts remain attributable.

3 Experiments

Fourteen experiments were conducted between 2026-06-28 and 2026-07-02 to map the reward and action-space design required for cloud-aware autonomous pointing. Each run tests one concrete design choice; the sequence is cumulative rather than a search for a single winning configuration. The arc divides into three phases: (i) diagnosing why naive training setups fail (Exps 1–5), (ii) reward shaping and action-space design (Exps 4–9), and (iii) MPO-specific stabilisation and further ablations (Exps 8–12), followed by scale and cadence probes (Exps 13–14). Experiment 6 (modular encoder rerun) was deferred and is not included.

Experimental protocol. A fixed random seed (7) ensures identical altitude draws, cloud layouts, and target positions across all compared runs. All experiments share the canonical simulation architecture described in Section 2.10.

Before training begins, *warmup episodes* pre-fill the replay buffer using the deterministic baseline policy. This was not the initial choice: early attempts using random torque commands or a zero-torque coast strategy yielded no reward signal, because neither strategy ever issues capture commands that earn positive credit. The baseline policy, by contrast, reliably captures targets on clear passes. Even so, only around ten capture actions fall within a 516-step episode ($\approx 2\%$ of all transitions), so the buffer is extremely sparse in rewarded transitions. The warmup data therefore serves a dual purpose: it bootstraps the replay buffer with at least some positive-credit transitions, and the episode return it achieves is the KPI comparison anchor used throughout.

Evaluation episodes run the current learned policy without gradient updates; eval return is the primary metric. A run is said to show a *learning signal* if its per-episode training return trends upward over training, indicating the policy has discovered the reward structure. Concretely, this requires at least one training episode to substantially exceed the warmup return and for the

trend to be sustained rather than a single outlier. Runs without a learning signal converge to a flat return plateau from episode 1 onward, regardless of the number of training episodes, and are reported as *not supported*. In the verdict column, *supported* means the design choice under test produced the expected learning or behavioural change (for example selective shuttering or a stable upward return trend); it does not by itself claim mission-ready outperformance of the baseline. Results are archived as JSON files per experiment; key metrics are cited directly below.

3.1 Master comparison table

Table 8 lists the full configuration and primary KPI for every run. The **baseline anchor** row is the deterministic baseline evaluated at matched seeds (mean eval return -38.6 ; anchored in Exp 13).

Table 8: Master experiment comparison. Algorithm: SAC = Soft Actor-Critic; MPO = Maximum a Posteriori Policy Optimisation. Action: T = torque (direct RW), V = vector pointing (nadir-relative offset). Reward flags: B = budget-exhausted shutter penalty; W = within-budget waste penalty; E = torque effort; S = safe-mode per-step penalty. Seeds: fixed 7 for all ML experiments; env draws are seeded per warmup fingerprint. Eval return is mean over 2 evaluation episodes unless otherwise noted. Δ is relative to the immediate predecessor configuration.

Exp	Config	Alg	Act	Reward flags	Warmup	Train	Eval eps	Eval return	Verdict
nb07	baseline	—	V	—	—	—	2	-38.6	—
1	mpo_t05	MPO	T	—	5	7	2	-101.6	not_supported
1	mpo_t09	MPO	T	—	5	7	2	-101.6	not_supported
2	sac_a0	SAC	T	—	5	7	2	-78.9	not_supported
2	sac_a1	SAC	T	—	5	7	2	-86.5	not_supported
3	compare_sac	SAC	T	—	5	$37^\dagger$	2	-22.2	supported
3	compare_mpo	MPO	T	dense	5	$22^\dagger$	2	-51.6	not_supported
4	ref0 (torque)	SAC	T	—	5	50	2	-11.1	partial
4	ref1 (vector)	SAC	V	—	5	50	2	-24.1	partial
5	mpo_s (90/140)	MPO	T	sparse	5	50	2	-51.6	not_supported
5	mpo_m (140/256)	MPO	T	sparse	5	50	2	-51.6	not_supported
5	mpo_l (256/512)	MPO	T	sparse	5	50	2	-51.6	not_supported
7	penalty_on	SAC	V	B	5	50	2	$+10.6$	supported
8	torque_sparse	MPO*	T	—	5	50	2	-81.1	partial
9	waste_off_budget_on	SAC	V	B (W off)	5	50	2	$+106.4$	supported
10	safe_mode_on	MPO*	T	S($k=2.0$)	5	50	2	-510.4	not_supported
11	vector_sparse	MPO*	V	—	5	50	2	$+45.5$	supported
12	vector_effort	MPO*	V	E($k=0.1$)	5	50	2	$+25.3$	not_supported
13	baseline_1_1_1	nb07	V	—	—	—	2	-38.6	—
13	cadence_1_1_10	MPO*	V	—	5	50	2	-46.4	not_supported
13	cadence_1_1_50	MPO*	V	—	5	50	2	-15.4	partial
14	multienv	MPO*	V	—	5	350	2	n/a ‡	not_supported

† Early abort (patience 20 eps). * MPO with decoupled-KL dual fix (Exp 8). ‡ Run crashed (OSError).

3.2 Exp 1: Shutter threshold ablation

Motivation. Preliminary runs showed the MPO agent emitting ~ 968 shutter commands per episode, far exceeding the budget of ten and firing the shutter on nearly every controller step. The hypothesis was that raising the continuous action threshold (above which a shutter command is issued) from 0.5 to 0.9 would mechanically reduce this saturation and, combined with a 15s credit window around each accepted capture, create a learnable reward gradient.

Config and result. Two configurations (shutter threshold 0.5 and threshold 0.9) were run for 7 training episodes at $\Delta t = 1.5$ s. Both produced an identical eval return of -101.6 with ~ 516 shutter commands per episode. The policy’s shutter action was saturated near its maximum

value (mean ≈ 0.999) at both thresholds. It fired the shutter on almost every step regardless of threshold setting. Neither showed a learning signal; every shutter command produced zero useful capture credit.

Verdict. Not supported. Threshold tuning is not an effective lever while the policy saturates the action dimension. The credit window creates latent mid-episode reward bursts but does not fix credit assignment.

3.3 Exp 2: Modular encoder ablation

Motivation. The controller observation is 105-dimensional with 50+50 structured bearing/mask fields. Flattening these into a single MLP may be sample-inefficient compared to a group-compressor [4]. Variant A1 replaced the flat concat with a Linear(50 $\rightarrow$ 8) compressor on each field group.

Config and result. SAC sparse, 7 training episodes. A0 (flat): eval -78.9 , 223 shutters/ep. A1 (compress): eval -86.5 , 98 shutters/ep. Neither variant showed a learning signal; both produced zero meaningful captures. A1’s critic loss diverged (3190 vs A0’s 1420 at episode 7), indicating training instability in the compressed branch.

Verdict. Not supported. The structured observation encoder shows no benefit in the 7-episode non-learnable slice. The encoder is deferred for a rerun once a stable learning configuration exists (deferred Exp 6).

3.4 Exp 3: SAC vs. MPO algorithm comparison

Motivation. Exps 1–2 both used MPO; learning failure could be algorithm-specific. Exp 3 runs SAC (sparse reward) and MPO (dense reward) under matched protocol to isolate the algorithm–reward interaction [1, 5].

Config and result. Early-abort patience of 20 episodes. SAC sparse (early-abort at ep 37): eval -22.2 , clear learning signal, best training return $+5.3$. MPO dense (early-abort at ep 22): eval -51.6 , no learning signal, KL divergence $\approx 3.8 \times 10^5$. SAC clearly separated from MPO: the SAC policy converged while MPO’s policy update blew up.

Verdict. Supported. SAC learns under sparse reward; MPO dense fails in this configuration. Algorithm choice matters; MPO’s failure traced to an unconstrained M-step dual, motivating the fix in Exp 8.

3.5 Exp 4: Vector pointing mode

Motivation. Outputting a direct reaction-wheel torque conflates two problems: target scheduling and rate control. The vector pointing interface delegates rate control to a proportional–derivative loop and exposes a nadir-relative offset to the policy, reducing the action space the learner must shape for scheduling.

Config and result. SAC sparse, 50 training episodes. Ref0 (torque): eval -11.1 , clear learning signal, best training return $+8.4$. Ref1 (vector): eval -24.1 , clear learning signal, best training return $+95.8$. Ref1’s eval was worse, but video of training episode 11 ($+95.8$) showed deliberate off-nadir scheduling and sparse mid-orbit shutter firings. This showed a qualitatively different and promising behaviour. End-of-episode shutter spam after budget exhaustion remained apparent in Ref1.

Verdict. Partial. Vector mode is valid and both variants learn, but the eval gap between them is unexplained. The shutter-budget ignorance motivates Exp 7. Vector pointing mode is retained as the preferred action interface for subsequent experiments.

3.6 Exp 5: MPO network width ablation

Motivation. Before investigating the MPO dual, the hypothesis that a 90/140-unit head was capacity-limited was tested with three widths: S (90/140), M (140/256), L (256/512). This was gated after the encoder experiment following de Bruin et al. [2] suggestions on structure-before-capacity.

Config and result. MPO sparse, 50 training episodes. All three widths produced identical eval -51.6 and identical KL freeze/explosion patterns. Post-episode-1 returns were flat; every shutter command produced zero useful capture credit across all three width variants.

Verdict. Not supported. Network width is not the bottleneck; the shared collapse pattern points to the learning algorithm.

3.7 Exp 7: SAC vector + budget-exhausted shutter penalty

Motivation. Exp 4 video (Ref1 ep 11) showed mid-orbit sparse shuttering but persistent shutter spam after the 10-capture budget was exhausted. The budget-exhausted penalty adds $-k_{\text{waste}}$ to the step reward whenever the agent commands a shutter with zero remaining budget (Section 2.7). Torque-path SAC with a static threshold (Exp 1) failed to stop spam; a direct negative reward on the offending action is the minimal correct lever.

Config and result. SAC sparse + vector + budget penalty, 50 training episodes. Eval $+10.6$ (vs Ref1 baseline -24.1 , $\Delta = +34.7$). Train shutter commands: 478/ep vs 7071/ep Ref1 (-93%). Train meaningful shutter fraction: 0.220 vs 0.015 Ref1 ($15\times$). Post-peak return std: 17.5 vs 28.4 (-38%).

Verdict. Supported. Budget-exhausted penalty is an effective design lever: shutter spam drops sharply and selective scheduling emerges. This reward term is kept for all subsequent experiments.

3.8 Exp 8: MPO decoupled-KL dual fix (torque mode)

Motivation. Exps 3 and 5 both showed MPO freezing or exploding at $\text{KL} \approx 3.8 \times 10^5$ or ≈ 0 . Investigation identified the root cause: the M-step used a single coupled Lagrange multiplier for both mean and variance KL constraints, instead of the two independent multipliers specified in Abdolmaleki et al. [1]. Exp 8 validates the corrected decoupled-KL dual in torque mode against the Exps 3/5 comparators.

Config and result. MPO (fixed dual) sparse torque, 50 training episodes, seed 7. Clear learning signal; eval -81.1 vs pre-fix floor -51.6 . Best train $+8.7$ (ep 28). KL converged to 0.021; temperature dual settled at 0.788. Train torque saturation fraction: 0.951 (vs ≈ 0.998 pre-fix). No meaningful eval captures.

Verdict. Partial. The algorithm fix is confirmed: bounded KL and rising train returns. Eval return remains below the baseline anchor; torque mode still saturates and safe-mode activates frequently, motivating Exps 10 and 11.

3.9 Exp 9: SAC shutter reward split (waste penalty off)

Motivation. Exp 7 had both the budget-exhausted penalty and the within-budget waste penalty active. Since applied-capture credit already scales with image quality and cloud fraction (Eq. 16), a second explicit waste penalty on the same action may over-constrain shutter exploration. Exp 9 tests turning off the waste penalty while keeping the budget-exhausted term.

Config and result. SAC sparse + vector + budget penalty (W off), 50 training episodes. Eval +106.4 (vs Exp 7 +10.6, $\Delta = +95.8$). Best train +169.9 (ep 36). Train meaningful fraction: 0.286 vs 0.220 (Exp 7). Eval meaningful fraction: 0.667 vs 0.400. Post-budget shutter commands: 75 (penalty still active).

Verdict. Supported. Relaxing the over-constraining within-budget waste penalty yields a large eval improvement. This is the best SAC configuration in the series and defines the reward stack used as the reference for later MPO comparisons.

3.10 Exp 10: MPO safe-mode penalty

Motivation. Exp 8 showed safe-mode activating 5–6 times per episode in torque mode with no reward signal. A per-step penalty during safe-mode recovery intervals ($k = 2.0$) was hypothesised to reduce activations and improve applied torque credit assignment.

Config and result. MPO (fixed dual) sparse torque + safe-mode penalty ($k = 2.0$), 50 training episodes. Eval -510.4 (vs Exp 8 -81.1 , catastrophic regression). Training return improved from -1168 (ep 0) to ≈ -350 (ep 23) but stagnated. Train shutter commands dropped from 153 (ep 0) to 2–7 (eps 30–49).

Verdict. Not supported. The penalty taught the agent to avoid all aggressive torque (including captures) rather than only avoiding safe-mode entry. Safe-mode activations co-occur with off-nadir pointing required for capture; the penalty suppressed both. Per-step safe-mode penalty at this coefficient is ruled out for torque-mode MPO. Vector mode (Exp 11) provides a structurally cleaner solution.

3.11 Exp 11: MPO fixed dual, vector mode

Motivation. Exp 10 ruled out reward-based safe-mode mitigation in torque mode. Exp 8 confirmed the dual fix; the natural next step is MPO in vector pointing mode, mirroring the Exp 4 SAC comparison but now with a working MPO agent. Vector mode avoids the safe-mode interference seen in torque mode.

Config and result. MPO (fixed dual) sparse vector, 50 training episodes, seed 7. Eval +45.5 (vs Exp 8 -81.1 , $\Delta = +126.6$). Best train +82.3 (ep 42). Positive returns from episode 7 onward; monotonic improvement from ep 5. KL: 0.014; η : 0.014 (vs 0.788 in torque mode). Shutter commands: 148 (ep 0) $\rightarrow$ 8–10 (eps 30–49). The policy learned selective shuttering.

Verdict. Supported. Vector mode resolves the torque-mode MPO stagnation: the first MPO configuration with positive eval return. The temperature dual settles much lower in vector mode, reflecting a tighter value distribution in the simplified action space. Vector pointing mode is retained for subsequent MPO experiments. The remaining gap to SAC Exp 9 (+106.4) suggests the MPO branch has not yet been tested with the same reward stack as Exp 9.

3.12 Exp 12: MPO vector + torque-effort penalty ablation

Motivation. The training workflow disables the torque-effort regulariser in vector mode by default. Exp 12 re-enables effort ($k = 0.1$) while keeping all other Exp 11 settings.

Config and result. MPO (fixed dual) sparse vector + torque effort ($k = 0.1$), 50 training episodes. Eval +25.3 (vs Exp 11 +45.5, $\Delta = -20.2$). Best train +75.6 (ep 20). Four consecutive zero-return episodes at eps 36, 41, 42, 43. KL: 0.013; η : 0.027.

Verdict. Not supported. Torque effort in vector mode hurts performance: the effort penalty targets the pointing-controller wheel command, not the policy’s pointing offset, creating a conflicting gradient that discourages off-nadir pointing irrespective of imaging context. Effort regularisation should remain disabled in vector mode.

3.13 Exp 13: MPO learning-cadence sweep

Motivation. The MPO agent performs one gradient update per environment step by default. If the agent learns faster than the environment provides signal, or if gradient noise from a partially filled replay buffer impedes early learning, a different cadence may help. Exp 13 evaluates three cadence profiles: update every step, every 10 steps, and every 50 steps.

Config and result (partial). MPO (fixed dual) vector sparse, 50 training episodes per configuration, 2 eval episodes. The deterministic baseline (comparison anchor): eval -38.6 . Update every 10 steps: eval -46.4 (worse than baseline). Update every 50 steps: eval -15.4 (best in sweep; still below Exp 11 +45.5).

Verdict. Partial / incomplete. Sparser gradient updates (every 50 steps) marginally outperform the per-step default in this short slice, but results remain below Exp 11 and the every-10-steps configuration falls below the comparison anchor (-38.6). The full cadence sweep was still in progress at report cutoff; a longer training slice is needed for a definitive conclusion.

3.14 Exp 14: MPO multi-environment target selection

Motivation. All prior experiments use a single environment seed (7) with a fixed target corridor. Generalisation to diverse orbital configurations, altitudes, and cloud draws requires training across multiple environment draws. Exp 14 attempted to run 350-episode multi-environment MPO to test this scaling hypothesis.

Config and result. MPO (fixed dual) vector sparse, 350 training episodes, multi-seed draws. The run crashed during Stage C with a filesystem error (likely a path-length limit on Windows for long run-directory names). Stage B partial training metrics were available (mean score 0.228) but are not directly comparable to the episodic eval return metric used in the rest of the series.

Verdict. Not supported (run failure; results inconclusive). Multi-environment training remains an open direction. Recommended follow-up: shorten run directory names, increase episode budget, and match the evaluation metric to the rest of the series.

4 Results

This section summarises what the experiment sequence revealed about reward shaping and action-space design. Eval return and shutter behaviour are reported against the deterministic

baseline anchor (-38.6); numbers above that anchor indicate a configuration learned useful scheduling on the fixed evaluation seed, not that the training problem is solved in general.

4.1 Aggregate performance comparison

Table 9 distils the closed experiments into a single snapshot of the design search. The reference columns are the **deterministic baseline anchor** (-38.6) and the **best SAC configuration** (Exp 9, $+106.4$ on seed 7).

Table 9: Aggregate results by experiment phase. Eval return is mean over 2 evaluation episodes on seed 7. $\Delta_{\text{base}} = \text{eval return} - (-38.6)$ baseline anchor. Shutter cmds/ep: train mean where available.

Exp	Alg	Key change	Eval return	Δ_{base}	Shutter cmds/ep (train)
Baseline anchor	—	Fixed schedule	-38.6	—	10 (fixed)
<i>Phase 1: Diagnosis</i>					
1	MPO	Threshold 0.5/0.9	-101.6	-63.0	516
2	SAC	Compressed encoder	-78.9	-40.3	236
3	SAC	Sparse reward	-22.2	$+16.4$	—
3	MPO	Dense reward	-51.6	-13.0	—
5	MPO	Width ablation (all)	-51.6	-13.0	—
<i>Phase 2: Action space & reward shaping</i>					
4	SAC	Torque mode (Ref0)	-11.1	$+27.5$	—
4	SAC	Vector mode (Ref1)	-24.1	$+14.5$	7071
7	SAC	Vector + budget penalty	$+10.6$	$+49.2$	478
9	SAC	Vector + budget, waste off	$+106.4$	$+145.0$	—
<i>Phase 3: MPO stabilisation & ablations</i>					
8	MPO	Decoupled-KL fix, torque	-81.1	-42.5	—
10	MPO	Safe-mode penalty ($k=2.0$)	-510.4	-471.8	2–7
11	MPO	Fixed dual, vector	$+45.5$	$+84.1$	8–10
12	MPO	Vector + effort ($k=0.1$)	$+25.3$	$+63.9$	—
<i>Phase 4: Scale & cadence</i>					
13	MPO	Cadence 1:1:50	-15.4	$+23.2$	—
14	MPO	Multi-env (crashed)	n/a	—	—

4.2 Learning progression

The episode-level learning curves show how the design space was narrowed:

- **Exps 1–5:** Naive configurations (threshold tuning, encoder variants, algorithm and width ablations) produce flat or degrading returns. No run in this phase crosses positive territory, confirming that the training problem is not solved by default.
- **Exp 3:** SAC with sparse reward is the first configuration to show a rising training trajectory, establishing that the environment is learnable once the reward and algorithm pairing is correct.
- **Exp 7:** Adding a budget-exhausted shutter penalty to vector-mode SAC yields the first positive eval return ($+10.6$) and cuts train shutter commands from ~ 7000 (Exp 4) to 478.

- **Exp 9:** Removing the within-budget waste penalty further improves eval return (+106.4). On eval episodes, two thirds of capture attempts are scientifically valid (non-cloud, within budget, previously uncaptured target).
- **Exp 11:** MPO in vector mode reaches +45.5, the first positive MPO eval return, after the decoupled-KL dual fix (Exp 8) removed the algorithm-level collapse.

Per-experiment learning curves are available in the run artefact directories.

4.3 Shutter spam and budget efficiency

Shutter command count per episode tracks whether a policy has learned selective scheduling rather than firing continuously. A well-trained policy would issue at most ten commands per episode (one per target within budget), timed to clear-sky windows. The deterministic baseline issues exactly ten commands but ignores cloud content, so an estimated 30–40% of its captures are wasted on cloud-blocked or empty targets.

Table 10 tracks the spam reduction path.

Table 10: Shutter spam reduction across the SAC vector experiment chain. All figures are training means unless noted.

Experiment	Cmds/ep (train)	Meaningful frac (train)	Notes
Overnight H1a (pre-Exp 1)	968	0	MPO, threshold 0.5
Exp 1 mpo_t05/t09	516	0	Threshold ineffective
Exp 4 Ref1 (SAC vector)	7071	0.015	Budget ignorance
Exp 7 (budget penalty)	478	0.220	−93% cmds vs Ref1
Exp 9 (waste off)	—	0.286 / 0.667 (eval)	Best scheduling
Exp 11 (MPO vector)	8–10	—	Budget-aware MPO
Baseline anchor (reference)	10	~0.60–0.70	Deterministic, cloud-blind

The budget-exhausted penalty (Exp 7) was the single most impactful reward change in the series: a 93% reduction in shutter commands with a simultaneous increase in eval return. Removing the waste penalty (Exp 9) added a further behavioural improvement on top of that configuration. Together, these results show that reward structure, not algorithm choice alone, determines whether selective shuttering emerges.

4.4 Safe-mode and safety overhead

In torque mode (Exps 1–5, 8) the safety controller activates 5–6 times per episode, locking out the agent for 60 s per event (Section 2.6). Each lockout consumes ≈ 40 controller steps with no capture opportunity. With 516 steps per episode and 5 lockouts, approximately 40% of the episode is unavailable to the agent.

Vector mode (Exps 4 Ref1, 7, 9, 11, 12) eliminates safe-mode lockouts in the experiment runs: the hold-last semantics prevent the pointing reference from exceeding θ_{hard} , so the safety controller is never triggered.

This structural difference explains much of the torque-versus-vector gap. The safe-mode penalty (Exp 10) attempted to replicate this improvement via reward shaping but produced the worst result in the series (−510.4), confirming that action-space redesign (vector mode) is preferable to reward penalties for aligning safety and learning.

4.5 Algorithm comparison: SAC vs. MPO

At matched protocol (sparse, vector, 50 episodes, seed 7), the best SAC configuration (Exp 9, +106.4) exceeds the best MPO configuration (Exp 11, +45.5) on eval return by 60.9 points. The gap is partially explained by reward configuration: Exp 9 uses the waste-off, budget-on reward while Exp 11 uses the budget-on, waste-on default. An MPO run with the Exp 9 reward settings was not completed within the project timeline.

Both best configurations exceed the baseline anchor on this seed (-38.6). That is encouraging for the design search but not sufficient evidence of robust mission-level superiority: evaluation uses only two episodes at a fixed seed, and several intermediate configurations never learned at all. The more durable finding is behavioural. Both algorithms can learn selective shuttering once reward and action-space choices are correct. That is a stronger claim than reliable baseline outperformance under all training conditions.

5 Discussion

5.1 What the experiment sequence taught

The motivating idea, use onboard vision to shutter only on clear targets, is simple to state but hard to train. The fourteen experiments were not a single attempt to beat the baseline; they were a structured search for reward and action-space configurations that make learning possible. Three lessons emerged.

1. Reward structure before algorithm. Exps 1–5 showed that neither threshold tuning, encoder architecture, algorithm choice, nor network width were sufficient on their own. The break came in Exp 3 (SAC learns under sparse reward) and was exploited in Exps 7 and 9, where budget-exhausted penalty and waste-penalty removal each unlocked further learning. Reward engineering, specifically what information is available at the action clock, was the dominant design lever.

2. Action-space design before reward fine-tuning. The comparison between torque and vector pointing modes (Exps 4, 8, 11) shows that delegating rate control to the onboard pointing controller is structurally important. Vector mode removes safe-mode lockouts in the experiment runs (Section 4.4), resolving a credit-assignment problem that no reward penalty could fix cleanly (Exp 10). This is a design result: the agent should command a pointing offset, not raw wheel torque, for cloud-aware scheduling in this environment (Section 5.3).

3. Algorithm stability as a prerequisite. MPO’s KL blow-up (Exps 3, 5) blocked all progress on the MPO branch until the root cause (an unconstrained M-step multiplier) was identified and corrected (Exp 8). The fix unlocked learning in vector mode (Exp 11) without any other design change. Implementation correctness can masquerade as algorithm failure across multiple experiments. That lesson applies before comparing reward or action-space variants.

5.2 Answer to the research question

Cloud-aware autonomous scheduling is not trivial to train in this simulation. A minimal viable training stack requires at least:

- a sparse capture-credit reward with a budget-exhausted shutter penalty;
- vector pointing mode (nadir-relative offset) rather than direct torque commands;

- a correctly implemented MPO trust-region update, or SAC as the more forgiving default;
- baseline-policy warmup to seed the replay buffer with rare positive-credit transitions.

With that stack, both SAC and MPO learn selective shuttering absent in the deterministic baseline. Several configurations that looked reasonable (threshold tuning, network width, safe-mode penalties, torque effort in vector mode) failed to produce learning or made it worse. Eval return on seed 7 exceeded the baseline anchor for the best SAC and MPO configurations, but the evaluation protocol is too narrow to claim robust mission superiority; the durable output of the semester is the mapped design space and the list of rejected shortcuts.

5.3 Control hierarchy and action interface

Development did not begin with the interface the experiments ultimately recommend.

From torque to safety arbitration. The first experimental interface exposed raw reaction-wheel torque to the learning agent. Unconstrained policies could command aggressive slews; in simulation this produced sustained rotation that would be unacceptable on flight hardware. Review with system engineers motivated treating attitude safety as non-negotiable: every agent command, whether torque or pointing request, passes through a safety controller that can taper, reject, or replace commands with a safe-mode recovery sequence (Section 2.6). That stack is shared by the deterministic baseline and all learned policies.

Action modes tested. Three action interfaces were explored:

- **Torque mode:** the policy commands wheel torque directly (Exps 1–5, 8, 10).
- **Vector mode:** the policy commands a nadir-relative pointing offset; the OBC converts it to PD torque (Exps 4, 7, 9, 11–13).
- **Target selection:** a higher-level discrete choice of which target to engage; explored prematurely in Exp 14 and not completed within the semester.

Segregation of control layers. The experiment arc supports separating what the agent should learn from what the OBC should execute. Learning precise torque profiles or continuous pointing offsets is not the primary scheduling problem in this environment: the agent must decide *when* to expose the shutter given cloud feedback, while rate control and maneuver execution belong downstream. Vector mode reflects that split and improved learning in practice (Exps 4, 11). A torque-trained agent might in principle discover rich low-level strategies, for example multiple captures within one forward-motion-compensation maneuver, but a scheduling policy that selects targets and shutter timing could achieve similar mission outcomes with simpler credit assignment. Energy-optimal wheel commands remain a distinct optimisation problem; they should be trained or tuned in a separate low-level loop rather than entangled with cloud-aware target scheduling.

Intended end state. The long-term architecture is a three-level stack: mission intent from the ground or onboard planner, a vision-driven scheduling policy (target and shutter decisions), and OBC execution with PD pointing and hard safety limits. This semester maps the middle layer under a simplified 2D simulator; the lowest layer is modelled but not co-trained for efficiency.

5.4 Rejected paths

Table 11 summarises design choices that were tested and ruled out.

Table 11: Rejected experimental directions with rationale.

Experiment	Design choice tested	Why rejected
—	MPC pointing controller	Superseded by onboard PD pointing; lower scope and comparable results
—	Latent-only dense reward	Dense reward tested at $10\times$ applied scale; did not unblock MPO
Exp 1	Shutter threshold $0.5 \rightarrow 0.9$	Policy saturates shutter output regardless of threshold
—	15 s capture credit window	Latent bursts visible; applied capture remains zero
Exp 5	MPO network width ablation	Identical collapse at all widths; capacity not the bottleneck
Exp 9	Within-budget waste penalty (on)	Redundant with apply-credit sparsity; penalising same failure mode twice
Exp 10	Safe-mode penalty	Correlation with aggressive pointing suppresses useful captures
Exp 12	Torque effort in vector mode	Pointing-controller torque $\neq$ policy action; conflicting gradient

5.5 Limitations

Two-dimensional simulation. The entire dynamics model, camera model, and reward signal operate in a single orbit plane. Real three-axis attitude control involves cross-coupling between roll, pitch, and yaw; disturbance torques from solar pressure, magnetic field, and gravity gradient; and three-dimensional off-nadir imaging geometry. The 2D model provides a tractable learning environment but the policies are not directly transferable to hardware.

Single-pass, single-orbit mission. Each episode models one overflight of a fixed 50-target corridor. Revisit scheduling, downlink windows, and multi-orbit memory are not represented. The learned policy has no access to observations from prior passes.

Simulation-only evaluation. No hardware-in-the-loop or software-in-the-loop tests were conducted. The camera model is a geometric ray tracer with an analytic smear formula; radiometric calibration, MTF, and sensor noise are absent. Generalisation from this environment to a real EO instrument is an open question.

Fixed-seed evaluation. All primary eval returns are computed on two episodes at seed 7. The train-to-eval gap visible in several experiments (e.g. Exp 9 train +169.9 vs eval +106.4) may partially reflect evaluation seed dependence. A robust evaluation across ≥ 10 seeds is recommended before citing the best numbers as reliable performance estimates.

Budget and episode length. The 10-capture budget over 516 steps is an engineered constraint that simplifies mission planning but may not represent realistic EO tasking where capture rate, data volume, and ground station contact are coupled objectives.

5.6 Deferred extensions

The following directions are documented as promising but were not completed within the project:

- **Modular encoder rerun (deferred Exp 6).** Once learning is established, the vector-compressor encoder variant from Exp 2 should be retested. The high-dimensional observation with structured bearing and mask fields is a candidate for entity-based architectures [4].
- **MPO with Exp 9 reward config.** Exp 11 reached +45.5 without the waste-penalty-off configuration used in Exp 9. A direct MPO run at Exp 9 reward settings would quantify the algorithm gap more cleanly.
- **Multi-environment training (Exp 14 rerun).** Training across diverse seeds, altitudes, and cloud draws is essential for deployment robustness. Exp 14 crashed on a Windows path-length limit; a rerun with corrected run-directory naming and 350+ episodes is recommended.
- **Three-axis attitude extension.** Moving from 2D to 3D dynamics is the prerequisite for any hardware-facing transfer. The simulation architecture supports this extension via replacement of the single-axis RW model with a 3-wheel assembly and attitude quaternion propagation.
- **Real imagery and radiometric reward.** Replacing the geometric smear quality metric with a model-based image quality score derived from sensor noise and scene contrast would couple the reward more directly to downstream science value.

6 Conclusion

We set out to answer a question that looks simple from the outside: if cloud-aware autonomous scheduling is obviously better than a fixed capture plan, what reward shaping and action-space design are needed to train such a policy in simulation? The semester work shows that the answer is not trivial.

A reproducible simulation and training platform was built to investigate that question systematically. Fourteen experiments varied reward structure, action representation, and learning algorithm under matched baseline conditions. Three design levers dominated the outcome: reward information at the action clock, delegating rate control to a pointing controller rather than raw torque commands, and correct implementation of the MPO trust-region update. Once those were in place, both SAC and MPO developed selective shuttering behaviour absent in the deterministic baseline. Reliable quantitative outperformance of that baseline on mission return was not established within the semester scope; the main contribution is a mapped design space and an experimental protocol for continuing the search.

Outlook. Near-term work: extend evaluation beyond aggregate return, tighten the protocol for longer training runs, and migrate control authority toward mission-level intent while keeping a single canonical episode artifact for train, eval, and render. Logical extensions beyond this semester include three-dimensional attitude and orbit propagation, richer terrain and real imagery ingestion, and multi-orbit scheduling.

A Supplementary material

A.1 Repository

The full source code, experiment scripts, and run artifacts are available at:

<https://github.com/cgriede/autonomous-satellite-control-eth-sem-proj>

A.2 Source index

Table 12 lists the high-level entry points for the components described in this report. All paths are relative to the repository root.

Table 12: High-level source index.

Component	Entry point
Earth & geodetic constants	<code>backend/environment_definition/constants/EARTH.py</code>
Mission profile (altitude, targets)	<code>backend/environment_definition/constants/MISSION.py</code>
Satellite inertia & camera optics	<code>backend/environment_definition/constants/SATELLITE.py</code>
Attitude safety thresholds	<code>backend/environment_definition/constants/ATTITUDE_SAFETY.py</code>
Simulation timing	<code>backend/environment_definition/constants/SIMULATION.py</code>
Reward weights	<code>backend/environment_definition/constants/AUTONOMOUS_CONTROL_REWARD.py</code>
Canonical simulation entry point	<code>backend/simulation/run_simulation.py</code>
Camera image quality model	<code>backend/simulation/image_quality.py</code>
Attitude safety controller	<code>backend/simulation/attitude_controller.py</code>
Deterministic baseline policy	<code>backend/s01_utils/baseline_overflight.py</code>
RL agent (MPO / SAC)	<code>backend/autonomous_control/controller_agent.py</code>
Training workflow	<code>backend/autonomous_control/training_workflow.py</code>
Experiment scripts	<code>backend/scripts/experiments/</code>

A.3 Live parameter record

Hyperparameters and constants are maintained in:

- `docs/presentation/environment-hyperparameters.md`
- `docs/presentation/machine-learning.md`
- `docs/presentation/technical-constants.md`
- `docs/presentation/physical-reference.md`

A.4 Artifact paths

Training outputs: `backend/autonomous_control/runs/<run-id>/`. Experiment results: `backend/scripts/exp`

References

- [1] Abbas Abdolmaleki, Jost Tobias Springenberg, Jonas Degraeve, Steven Bohez, Yuval Tassa, Daniele Belov, Nicolas Heess, and Martin Riedmiller. Maximum a posteriori policy optimisation. *arXiv preprint arXiv:1806.06920*, 2018.
- [2] Tim de Bruin, Jens Kober, Karl Tuyls, and Robert Babuska. Integrating state representation learning into deep reinforcement learning. *IEEE Robotics and Automation Letters*, 3(3): 1394–1401, 2018.

- [3] Benedetta Ferrari, Jean-François Cordeau, Maxence Delorme, Manuel Iori, and Roberto Orosei. Satellite scheduling problems: A survey of applications in Earth and outer space observation. *Computers & Operations Research*, 171:106875, 2024. doi: 10.1016/j.cor.2024.106875.
- [4] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. On embeddings for numerical features in tabular deep learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [5] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.